KARAMANOĞLU MEHMETBEY ÜNİVERSİTESİ

FEN BİLİMLERİ ENSTİTÜSÜ

BİLGİSAYAR MÜHENDİSLİĞİ ANABİLİM DALI

BÜYÜK VERİYE GİRİŞ DERSİ VİZE ÖDEVİ

Apache Spark MLlib ile Büyük Ölçekli Uçuş Gecikmesi Verisi Üzerinde İkili Sınıflandırma Modelinin Geliştirilmesi ve Performans Optimizasyonu

Hazırlayan: Talha KIRCA

Numara: 250805026

Araştırma Konuları:

1) Hadoop'tan Spark'a Geçiş Nedenleri

2) Spark MLlib ile Büyük Veri Üzerinde Makine Öğrenmesi

Ders: Büyük Veriye Giriş

Öğretim Üyesi: Dr. Öğr. Üyesi Hüseyin ELDEM

Teslim Tarihi: Nisan 2026

ÖZET

Bu çalışmada, ABD Ulaştırma Bakanlığı'na (U.S. Department of Transportation) ait 2015 yılı uçuş gecikmesi veri seti üzerinde Apache Spark'ın dağıtık makine öğrenmesi kütüphanesi olan MLlib kullanılarak ikili sınıflandırma (binary classification) problemi ele alınmıştır. Çalışmanın amacı, bir uçuşun 15 dakikadan fazla rötar yapıp yapmayacağını önceden tahmin edebilen bir model geliştirmektir. Toplam 5.819.079 satır ve 31 sütundan oluşan ham veri seti PySpark ile okunmuş; eksik değer analizi, veri temizleme, özellik mühendisliği ve sınıf dengesizliği çözümü aşamalarından geçirilerek modelleme için hazır hâle getirilmiştir. Kategorik değişkenlerin yüksek kardinalitesi (~620 benzersiz havalimanı kodu) nedeniyle Google Colab ortamında karşılaşılan bellek taşması problemi, *Top-N kategori gruplaması* ve *ağaç tabanlı modeller için OneHotEncoder atlamalı boru hattı* stratejileriyle çözülmüştür. Üç farklı algoritma (Lojistik Regresyon, Random Forest, Gradient Boosted Trees) karşılaştırmalı olarak eğitilmiş; en yüksek başarı AUC=0,686 ve F1=0,687 (optimum karar eşiği ile) değerleriyle GBT modelinden elde edilmiştir. Sonuçlar, sınıf dengesizliği düzeltilmiş büyük ölçekli tablosal verilerde topluluk (ensemble) yöntemlerinin tekli modellere göre belirgin üstünlük sağladığını doğrulamaktadır.

Anahtar Kelimeler: Büyük Veri, Apache Spark, Spark MLlib, PySpark, İkili Sınıflandırma, Uçuş Gecikmesi Tahmini, Sınıf Dengesizliği, Özellik Mühendisliği, Gradient Boosted Trees.

İÇİNDEKİLER

1. GİRİŞ 1
1.1. Problem Tanımı 1
1.2. Veri Seti ve Araçlar 1
2. LİTERATÜR TARAMASI 2
2.1. Spark MLlib ile Büyük Veri Üzerinde Makine Öğrenmesi 2
2.2. Hadoop'tan Spark'a Geçiş Nedenleri 5
3. YÖNTEM 8
3.1. Geliştirme Ortamı ve Spark Konfigürasyonu 8
3.2. Veri Okuma ve Keşifsel Analiz 9
3.3. Eksik Veri Analizi ve Temizleme 10
3.4. Özellik Mühendisliği 11
3.5. Sınıf Dengesizliği ve Alt-Örnekleme 13
3.6. Çift-Pipeline Mimarisi 14
3.7. Kullanılan Sınıflandırma Modelleri 15
4. OPTİMİZASYON VE GELİŞTİRMELER 16
5. BULGULAR VE SONUÇLAR 18
5.1. Keşifsel Veri Analizi Bulguları 18
5.2. Model Performans Karşılaştırması 20
5.3. Özellik Önemi Analizi 22
5.4. Karar Eşiği Optimizasyonu 23
6. TARTIŞMA 24
7. SONUÇ 26
KAYNAKÇA 27

1. GİRİŞ

1.1. Problem Tanımı

Havayolu endüstrisi, küresel ekonomik faaliyetin en önemli bileşenlerinden birini oluşturmaktadır. Bu endüstride meydana gelen uçuş gecikmeleri; sadece yolcu memnuniyetsizliğine değil, aynı zamanda ciddi ekonomik kayıplara ve operasyonel zorluklara yol açmaktadır. Federal Aviation Administration (FAA) tarafından yayımlanan raporlar, yalnızca ABD'deki uçuş gecikmelerinin yıllık toplam maliyetinin 30 milyar doları aştığını göstermektedir. Dolayısıyla, bir uçuşun rötar yapıp yapmayacağını önceden tahmin edebilen güvenilir modellerin geliştirilmesi, havayolu şirketlerinin planlama süreçleri için kritik öneme sahiptir.

Bu çalışmada tanımlanan problem, aşağıdaki biçimde formüle edilebilir: *Bir uçuşun kalkış tarihi, havayolu şirketi, kalkış/varış havalimanı, planlanan kalkış saati ve mesafe bilgileri kullanılarak, uçuşun 15 dakikadan fazla rötar yapıp yapmayacağı nasıl tahmin edilebilir?* Problem, doğası gereği bir **ikili sınıflandırma (binary classification)** problemidir. Ancak veri setinin boyutu (5.8 milyon kayıt) ve sınıfların dengesiz dağılımı (~%82 zamanında, ~%18 rötar) nedeniyle klasik tek-makine makine öğrenmesi kütüphaneleri (örneğin Scikit-Learn) yetersiz kalmakta; **dağıtık hesaplama** gerektiren büyük veri araçlarına ihtiyaç duyulmaktadır (Meng ve ark., 2016).

1.2. Veri Seti ve Kullanılan Araçlar

Çalışmada kullanılan veri seti, ABD Ulaştırma Bakanlığı'nın (U.S. Department of Transportation, DOT) 2015 yılı için yayımladığı *Flight Delays and Cancellations* veri setidir. Veri seti, Kaggle platformu üzerinden *usdot/flight-delays* tanımlayıcısıyla erişime açılmıştır. Toplam **5.819.079 satır** ve **31 sütundan** oluşan bu veri seti; ay, gün, havayolu kodu, kalkış ve varış havalimanları, planlanan/gerçekleşen kalkış ve varış zamanları, mesafe, gecikme süreleri ve gecikme nedenleri gibi değişkenleri içermektedir.

Geliştirme ortamı olarak **Google Colab**; veri işleme ve modelleme için **Apache Spark 3.5 (PySpark API)** ve **Spark MLlib** kütüphanesi kullanılmıştır. Görselleştirme ve ikincil analiz için Pandas, Matplotlib, Seaborn ve Scikit-Learn yardımcı kütüphaneleri tercih edilmiştir. Spark oturumu 8 GB sürücü belleği, adaptif sorgu yürütme (Adaptive Query

Execution) ve 8 shuffle partisyonu ile yapılandırılmıştır. Tüm kod, yeniden üretilebilirliği sağlamak amacıyla açık kaynak olarak GitHub üzerinde paylaşılmıştır.

Bu raporun geri kalanı şu şekilde düzenlenmiştir: Bölüm 2'de seçilen iki konuya ilişkin ayrıntılı literatür taraması sunulmuştur. Bölüm 3'te uygulanan yöntem adım adım açıklanmıştır. Bölüm 4'te proje sürecinde yapılan optimizasyonlar ve özgün katkılar tartışılmıştır. Bölüm 5'te elde edilen bulgular grafikler ve tablolarla sunulmuş; Bölüm 6'da bu bulgular yorumlanmıştır. Rapor, Bölüm 7'deki sonuç ve kaynakça ile sonlandırılmıştır.

2. LİTERATÜR TARAMASI

Bu bölümde, çalışmanın araştırma boyutunu oluşturan iki konu — Spark MLlib ile büyük veri üzerinde makine öğrenmesi ve Hadoop ekosisteminden Apache Spark'a geçişin nedenleri — literatür temelli olarak ayrıntılı bir biçimde incelenmiştir. Seçilen iki konu birbirini tamamlayıcı niteliktedir: Bir yandan Spark'ın neden günümüzde endüstri standardı haline geldiği (geçiş nedenleri) tartışılmış, diğer yandan Spark ekosisteminin makine öğrenmesi bileşeninin (MLlib) mimarisi ve literatürdeki yeri ele alınmıştır.

2.1. Spark MLlib ile Büyük Veri Üzerinde Makine Öğrenmesi

2.1.1. Genel Bakış ve Motivasyon

Geleneksel makine öğrenmesi kütüphaneleri (örneğin Scikit-Learn), eğitim verisinin tek bir makinenin RAM'ine sığmasına bağımlıdır. Veri boyutu milyonlarca satıra ya da terabayt seviyelerine ulaştığında bu kütüphaneler bellek taşması nedeniyle çalışamaz hâle gelmektedir (Meng ve ark., 2016). İşte Spark MLlib, bu darboğazı aşmak amacıyla tasarlanmış **dağıtık (distributed)** bir makine öğrenmesi kütüphanesidir. MLlib; lojistik regresyon, rastgele orman (Random Forest), gradyan artırmalı ağaçlar (Gradient Boosted Trees) ve k-ortalamalar (k-Means) gibi klasik algoritmaları, veriyi bir kümedeki (cluster) onlarca veya yüzlerce sunucuya bölerek paralel olarak eğitecek biçimde yeniden tasarlamıştır.

Meng ve ark. (2016), Spark'ın açık kaynaklı dağıtık ML kütüphanesini kapsamlı biçimde tanıtan çalışmada; MLlib'in sınıflandırma, regresyon, kümeleme, işbirlikçi filtreleme ve boyut indirgeme gibi öğrenme ayarlarına yönelik hızlı ve ölçeklenebilir algoritmalar sunduğunu vurgulamıştır. Çalışmada, MLlib'in Scala temelli yerel lineer cebir kütüphaneleri (özellikle Breeze ve netlib-java) kullanılarak yürütüldüğü; Java, Scala ve Python API'leri aracılığıyla Spark ekosistemine entegre edildiği belirtilmiştir. Aynı çalışmaya göre Spark'ın

yinelemeli hesaplama yapısı büyük ölçekli makine öğrenmesi uygulamaları için idealdir ve MLlib'in sürekli gelişimini destekleyen geniş bir katılımcı topluluğu bulunmaktadır (Meng ve ark., 2016).

2.1.2. MLlib Mimarisi ve Boru Hattı (Pipeline) Kavramı

Apache Spark, veri işleme motoru olarak 2010'da UC Berkeley AMPLab'te geliştirilmiş ve iteratif hesaplamalar için optimize edilmiştir (Zaharia ve ark., 2012). Spark'ın çekirdeğinde **DAG (Directed Acyclic Graph — Yönlendirilmiş Asiklik Grafik)** tabanlı bir yürütme motoru yer alır; kullanıcı kodu önce SparkContext (veya modern API ile SparkSession) üzerinden bir plan olarak toplanır, bu plan Catalyst Optimizer ile iyileştirilir ve paralel görevler hâlinde çalışan Executor'lara dağıtılır. Her Executor, JVM içinde çalışan bir süreç olup kendi task'larını yürütür ve veriyi bellek içinde tutabilir; böylece disk tabanlı MapReduce'a göre önemli performans artışı sağlanır.

Spark MLlib'in mimarisinde öne çıkan kavramlardan biri **Pipeline (boru hattı)** yaklaşımıdır. Pipeline API, veri ön işleme (StringIndexer, OneHotEncoder, VectorAssembler), model eğitimi (örneğin LogisticRegression) ve değerlendirme aşamalarını tek bir akışta zincirleyerek kod karmaşasını azaltır ve modellerin üretim (production) ortamına alınmasını kolaylaştırır. Ayrıca Pipeline, eğitim verisi üzerinde fit edildiğinde elde edilen ön işleme parametrelerinin (örneğin StringIndexer'ın öğrendiği kategori eşlemeleri) test verisine transform olarak uygulanmasını sağlayarak **veri sızıntısı (data leakage)** riskini ortadan kaldırır. Bu özellik, akademik yayınlarda MLlib'in en güçlü yönlerinden biri olarak defalarca vurgulanmıştır (Meng ve ark., 2016; Apache Spark Documentation, 2025).

MLlib, RDD tabanlı (spark.mllib) ve DataFrame tabanlı (spark.ml) iki farklı API versiyonu sunmaktadır. Spark 2.x ile birlikte DataFrame tabanlı spark.ml birincil API hâline gelmiş; RDD tabanlı spark.mllib ise bakım (maintenance) moduna alınmıştır. Bu çalışmada da modern ve önerilen yaklaşım olan spark.ml (DataFrame tabanlı) API tercih edilmiştir.

2.1.3. Performans ve Ölçeklenebilirlik Çalışmaları

Theodorakopoulos vd. (2025), Spark MLlib üzerinde büyük ölçekli modellerin performansını analiz eden yakın tarihli bir çalışma gerçekleştirmiştir. K-Means, Random Forest ve Word2Vec gibi operatörlerin uygulamasında toplanan ölçümlerle oluşturulan tahmin modelleri, hiperparametre optimizasyonuyla iş akışlarını iyileştirmiştir. Çalışmanın en dikkat çekici bulgusu, uygun parametre ayarlamalarıyla MLlib iş akışlarında **%30'a varan**

kaynak tasarrufu ve **%25'e varan işlem süresi azalması** elde edilebildiğidir (Theodorakopoulos vd., 2025). Bu bulgular, büyük veri projelerinde yalnızca algoritma seçiminin değil, hiperparametre ayarlamasının da kritik önemde olduğunu göstermektedir.

Mezati ve Aouria (2026), Spark MLlib'i Apache Flink'in makine öğrenmesi kütüphanesi olan Flink-ML ile sistematik biçimde karşılaştırmıştır. Sonuçlar, Spark MLlib'in toplu işleme (batch processing) ve mikro-yığın (micro-batched) akış işlemlerinde optimize olduğunu; gerçek zamanlı akış odaklı uygulamalarda ise Flink'in avantajlı olabileceğini göstermiştir. Ölçümlerde Spark MLlib'in eğitim süresi (4006,4 s) ile Flink-ML'in süresi (4003,2 s) hemen hemen aynı bulunmuş; doğruluk açısından Flink-ML'in marjinal biçimde daha yüksek sonuçlar verdiği, ancak çıkarım hızının (inference throughput) Spark lehine bir parça daha iyi olduğu rapor edilmiştir (Mezati ve Aouria, 2026).

Güvenlik alanında Atefinia ve Ahmadi (2022), Spark MLlib algoritmalarını siber saldırı tespiti üzerine uygulamıştır. Yedi düğümlü bir Spark kümesinde Lojistik Regresyon, Destek Vektör Makineleri (SVM), Karar Ağaçları ve Naive Bayes algoritmaları kullanılarak bir intrüzyon veri seti üzerinde modeller eğitilmiş ve %100'e varan sınıflandırma doğruluğuna ulaşılmıştır. Bu çalışma, Spark MLlib'in paralel olarak çalıştırılabilirliğini ve büyük veri siber güvenlik uygulamalarında yüksek başarı sağlayabileceğini ortaya koymaktadır (Atefinia ve Ahmadi, 2022).

Spark MLlib'in öne çıkan özelliği **ölçeklenebilirliktir**. Dağıtık işlemler CPU ve bellek açısından tasarruf sağlarken, veri büyüklüğü arttıkça paralel düğümler sayesinde toplam eğitim süresi lineer değil alt-lineer (sub-linear) biçimde artmaktadır. Performansın bir diğer boyutu **bellek-içi hesaplama avantajıdır**: Spark, veriyi RAM'de tutarak klasik Hadoop MapReduce uygulamalarında görülen disk I/O yükünü büyük ölçüde azaltır; bu durum özellikle tekrar eden eğitim döngülerinde (iteratif algoritmalar) büyük hız kazancı sağlar. Literatürde yapılan kıyaslamalarda Spark MLlib'in Hadoop tabanlı çözümlere göre genelde 10 kat civarında daha hızlı sonuç alabildiği rapor edilmektedir (Tekdoğan ve Çakmak, 2021).

2.1.4. Uygulama Alanları ve Bu Çalışmadaki Yeri

Spark MLlib'in uygulama yelpazesi oldukça geniştir. Twitter verileri üzerinde yapılan duygu analizi çalışmalarında Lojistik Regresyon, SVM ve Naive Bayes modelleri başarıyla eğitilmiş ve Spark'ın bellek içi işleme avantajı sayesinde çoklu algoritmalarla paralel eğitim mümkün kılınmıştır (DergiPark, 2023). COVID-19 salgını sırasında yapılan dağıtık regresyon modellerinde Gradient Boosted Tree Regression (GBTR) ve Random Forest Regression

algoritmalarının iteratif süreçlerinin Spark kümeleriyle verimli biçimde yürütüldüğü; Spark MLlib'in tek makine Python kütüphanelerine göre daha düşük çalışma zamanı sağladığı gözlemlenmiştir (Heller, 2016).

Bu ödev kapsamında geliştirilen projede, 5,8 milyon satırlık ABD DOT uçuş verisi Spark MLlib kullanılarak işlenmiştir. Büyük veriye özgü tipik sorunlar — yüksek kardinaliteli kategorik değişkenlerin patlaması, sınıf dengesizliği, bellek kısıtı altında model eğitimi — gerçek bir senaryoda deneyimlenmiş ve alt-örnekleme (undersampling), Top-N kategori gruplaması ve çift-pipeline mimarisi gibi yaklaşımlarla çözüme kavuşturulmuştur. Çalışma, üç farklı sınıflandırma algoritmasını (Logistic Regression, Random Forest, Gradient Boosted Trees) karşılaştırmalı olarak değerlendirmekte ve literatürdeki bulgularla uyumlu biçimde topluluk yöntemlerinin üstünlüğünü doğrulamaktadır.

2.2. Hadoop'tan Spark'a Geçiş Nedenleri

2.2.1. Hadoop Ekosistemi ve MapReduce Modeli

Büyük veri ekosisteminin temelleri Apache Hadoop ile atılmıştır. Hadoop, Google'ın MapReduce ve GFS makalelerinden esinlenilerek 2005 yılında Doug Cutting ve Mike Cafarella tarafından geliştirilmiştir. Hadoop Distributed File System (HDFS), Hadoop'un dağıtık depolama katmanı olup büyük dosyaları küçük bloklara (varsayılan 128 MB) bölerek küme genelinde çoğaltır; bu sayede hem ölçeklenebilirlik hem de hata toleransı sağlanır (Singh ve Singh, 2024). HDFS mimarisi, tek bir **NameNode** (dosya sistemi meta-verisini yöneten ana sunucu) ve birden çok **DataNode** (veri bloklarını tutan işçi sunucular) üzerinde çalışır. Hadoop 2.x sürümüyle eklenen **YARN (Yet Another Resource Negotiator)**, küme kaynak yönetimini ve iş zamanlamasını üstlenerek Hadoop'u yalnızca MapReduce'a değil, Spark ve Flink gibi farklı çerçevelere de ev sahipliği yapabilen bir platforma dönüştürmüştür.

Hadoop'un işlem modeli MapReduce iki aşamalıdır: **Map** aşamasında veri anahtar-değer çiftlerine bölünüp işlenir; **Reduce** aşamasında ise aynı anahtara sahip sonuçlar birleştirilir. Her ara aşamada verilerin HDFS'e (diske) yazılması, MapReduce'un yoğun bir disk I/O yapmasına neden olur. Bu durum özellikle iteratif algoritmalar (örneğin gradyan inişi tabanlı makine öğrenmesi modelleri) için ciddi performans sorunlarına yol açmaktadır.

2.2.2. Apache Spark'ın Ortaya Çıkışı ve Temel Mimarisi

Apache Spark, 2012'de AMPLab (UC Berkeley) tarafından Hadoop'un bu sınırlamalarına çözüm olarak geliştirilmiştir (Zaharia ve ark., 2012). Spark'ın temeli, verileri **bellek içinde (in-memory)** işleyerek disk I/O'yu minimize etmek üzerine kuruludur. Spark, **Resilient Distributed Dataset (RDD)** adı verilen değiştirilemez (immutable) veri koleksiyonlarıyla çalışır; bu RDD'ler kümedeki işçiler arasında bölünür, hataya dayanıklı bir soy (lineage) bilgisi tutulur ve paralel işlenir. Zamanla Spark, RDD'lerin üstünde daha güçlü soyutlamalar sunmuştur: *DataFrame* (şemalı veri tablosu) ve *Dataset* (tip güvenli DataFrame) API'leri, Spark SQL sorguları ve Catalyst optimizasyonu ile SQL benzeri işlemleri kolaylaştırır.

Spark mimarisinde bir **Driver** (sürücü) programı ana akışı yönetir. Bu sürücü, kullanıcı kodunu DAG olarak organize eder ve görevleri YARN, Mesos, Kubernetes veya Spark'ın kendi standalone yöneticisi aracılığıyla **Executor** (işçi) süreçlerine dağıtır. Executor'lar, her biri üzerinde bellek havuzu ve iş parçacıkları barındıran JVM süreçleridir. Spark; HDFS, Amazon S3 ve Azure Blob gibi çeşitli depolama çözümleriyle entegre olabildiğinden, mevcut Hadoop altyapılarında veya bulut platformlarında kolayca kullanılabilir.

2.2.3. Geçişi Tetikleyen Temel Faktörler

Hadoop MapReduce'tan Apache Spark'a geçişin literatürde vurgulanan üç temel nedeni şunlardır:

- **Disk Tabanlı İşlemden Bellek İçi (In-Memory) İşlemeye Geçiş:** Hadoop MapReduce her bir işlem adımında veriyi diske yazar ve tekrar okur (yoğun disk I/O). Bu durum özellikle iteratif algoritmalarda devasa zaman kayıplarına yol açmaktadır. Spark ise RDD ve bellek içi işleme mimarisi sayesinde veriyi RAM üzerinde tutarak MapReduce'a kıyasla bellekte 100 kata, diskte ise 10 kata kadar daha hızlı performans sunabilmektedir (AWS, 2025; Soysal, 2025).
- **Tembel Değerlendirme (Lazy Evaluation) ve DAG Mimarisi:** Spark, yazılan kodu anında yürütmek yerine bir Yönlendirilmiş Asiklik Grafik (DAG) oluşturur. Sadece bir eylem (action) tetiklendiğinde tüm süreci en optimize yoldan tek seferde hesaplar. Bu yaklaşım, MapReduce'un adım-adım katı yürütme modeline göre önemli optimizasyon fırsatları tanır.
- **Bütünleşik Ekosistem:** Hadoop ortamında SQL sorguları için Hive, makine öğrenmesi için Mahout, gerçek zamanlı veri akışı için Storm gibi farklı araçlara ihtiyaç duyulurken; Spark, SQL (Spark SQL), gerçek zamanlı veri (Spark Streaming),

makine öğrenmesi (MLlib) ve graf analizi (GraphX) gibi alt bileşenleri tek bir çatı altında birleştirerek geliştirme sürecini ciddi oranda basitleştirir (Singh ve Singh, 2024).

2.2.4. Literatürdeki Karşılaştırmalı Çalışmalar

Tekdoğan ve Çakmak (2021) ICCBDC 2021 bildirilerinde, Hadoop MapReduce ve Spark'ın performansını sınıflandırma problemleri üzerinde karşılaştırmıştır. Sonuçlara göre Spark, eğitim aşamasında MapReduce'a göre yaklaşık 5 kat daha hızlı çalışmıştır. Ancak giriş veri hacmi arttıkça Spark'ın performansı görece düşmekte, ilave düğümlerin eklenmesiyle düzelme sağlanmaktadır. İlgi çekici bir bulgu olarak MapReduce'un sınıflandırma doğruluğunda Spark'tan yaklaşık %3 daha yüksek sonuç verdiği gözlemlenmiştir; bu durum, hız ile doğruluk arasındaki geleneksel ödünleşme (trade-off) ilişkisine işaret etmektedir (Tekdoğan ve Çakmak, 2021).

Singh ve Singh (2024), IRJET dergisinde yayımlanan çalışmalarında Hadoop ve Spark'ın yetenek, performans, ölçeklenebilirlik ve kullanım kolaylığı yönlerini sistematik biçimde değerlendirmiştir. Çalışma Hadoop'u HDFS ve MapReduce tabanlı depolama-işleme çerçevesi; Spark'ı ise bellek-içi, RDD tabanlı dağıtık işlem motoru olarak tanımlamıştır. Makale, Spark'ın mimarisi, işleme hızı, ölçeklenebilirlik ve kullanım kolaylığı avantajlarına dikkat çekerken; Hadoop'un büyük veri depolama ve toplu işlem iş yüklerinde olgun ve güvenilir bir platform olduğunu vurgulamıştır (Singh ve Singh, 2024).

Amazon Web Services'in resmi dokümantasyonunda (AWS, 2025), Hadoop'un veriyi toplu işlerken yüksek gecikmeye neden olan disk tabanlı I/O yaptığı, Spark'ın ise veriyi önce RAM'e çekerek çok daha hızlı işlediği belirtilmiştir. Hadoop her işlem aşamasında diske yazarken Spark ara sonuçları bellek içinde tutarak disk kullanımını minimize eder. Kaynakta ayrıca Spark'ın RAM kullanımına bağlı maliyetinin, Hadoop'un disk tabanlı uygun maliyetli ölçeklenebilirliğine karşı bir ödünleşme oluşturduğu da not edilmiştir.

Databricks'in 2026 yılı geçiş webinar'ında, başarılı bir Hadoop'dan Spark'a geçiş için beş adımlı bir yol haritası önerilmektedir: (1) Veri ve meta-verilerin eşzamanlı biçimde yeni platforma aktarımı, (2) Hive ve MapReduce kodunun Spark SQL/PySpark koduna dönüştürülmesi, (3) Mevcut güvenlik ve erişim kontrol sistemlerinin (örneğin Apache Ranger) yeni ortama uyarlanması, (4) Birleşik veri platformunun yönetimi ve DevOps süreçlerinin merkezileştirilmesi, (5) BI ve SQL araçlarının bulut ortamında kullanılabilir hâle getirilmesi

(Databricks, 2026). Bu adımlar, pratik geçiş sürecinin çerçevesini çizmekte ve geçiş kararının iş etkisi değerlendirmesini kolaylaştırmaktadır.

2.2.5. Özet Değerlendirme

Literatür genel olarak Spark'ın bellek içi işleme ile sağladığı **5-100 kat arasında değişen hız avantajını**, Hadoop ile karşılaştırıldığında en belirgin üstünlük olarak işaret etmektedir. Ancak bu üstünlük, Hadoop'un olgun hata toleransı, düşük donanım maliyeti ve HDFS'in dayanıklı depolama yetenekleriyle dengelenmektedir. Günümüzde modern büyük veri mimarilerinde sıklıkla karşılaşılan yaklaşım, Spark işlerini HDFS üzerinde depolanan veriler üzerinde YARN aracılığıyla çalıştırmak biçimindeki **hibrit mimarilerdir**. Bu çalışma kapsamında da Spark'ın bellek içi işleme ve MLlib pipeline avantajları yoğun biçimde kullanılmıştır; ancak Google Colab gibi tek-düğümlü bir ortamda çalışıldığı için dağıtık mimarinin tam potansiyeli (yatay ölçeklendirme) deneyimlenememiştir. Bu durum, Bölüm 6'da sınırlamalar başlığı altında ayrıntılı biçimde tartışılmıştır.

3. YÖNTEM

Bu bölümde, ABD Ulaştırma Bakanlığı'na ait büyük ölçekli uçuş gecikmesi veri setinin bir makine öğrenmesi modeline nasıl hazırlandığı ve uygulanan tüm aşamalar adım adım detaylandırılmıştır. Geliştirme sürecinin tamamı, ödev yönergesinde belirtilen gereksinime uygun olarak Google Colab ortamında, PySpark API'si üzerinden Apache Spark dağıtık mimarisi kullanılarak gerçekleştirilmiştir.

3.1. Geliştirme Ortamı ve Spark Konfigürasyonu

Çalışma, Google Colab'ın ücretsiz runtime'ı üzerinde yürütülmüştür. Ücretsiz Colab; 12,7 GB RAM, yaklaşık 2 CPU çekirdeği ve 107 GB geçici disk kaynakları sunmaktadır. Apache Spark oturumu, Colab'ın sınırlı belleğine uygun olacak şekilde aşağıdaki özel parametrelerle yapılandırılmıştır:

- **spark.driver.memory = 8g:** Driver process için 8 GB RAM rezerve edilmiştir. Bu, tek-düğümlü ortamda hem driver hem executor işlerinin yürütülmesine olanak tanır.
- **spark.driver.maxResultSize = 2g:** toPandas() ve collect() gibi işlemlerle driver'a aktarılabilecek maksimum sonuç boyutu 2 GB ile sınırlandırılmıştır; bu, kazara yapılan büyük veri transferlerinin Colab'i çökertmesini önler.

- **spark.sql.shuffle.partitions = 8:** Shuffle işlemlerinde (groupBy, join vb.) oluşturulacak partisyon sayısı varsayılan 200'den 8'e düşürülmüştür; 2 çekirdekli Colab ortamında 200 partisyon gereksiz ek yük (overhead) oluşturmaktadır.
- **spark.sql.adaptive.enabled = true:** Spark 3.x ile gelen **Adaptive Query Execution (AQE)** özelliği etkinleştirilmiştir. AQE, çalışma zamanında sorgu planını optimize eder ve asimetrik (skewed) veri dağılımlarında otomatik olarak partisyon birleştirme yapar.

Bu ayarlar, Colab'ın sınırlı kaynakları içinde Spark'tan maksimum verimi almayı hedefleyen ve akademik literatürde de önerilen (Theodorakopoulos vd., 2025) konfigürasyonlardır.

3.2. Veri Okuma ve İlk Yapısal İnceleme

Kaggle API aracılığıyla indirilen 572 MB boyutundaki *flights.csv* dosyası, spark.read.csv() metodu ile schema inference açık olarak (inferSchema=True) bir Spark DataFrame'e yüklenmiştir. Ham veri setinin satır sayısı 5.819.079 ve kolon sayısı 31 olarak doğrulanmıştır (Şekil 1).

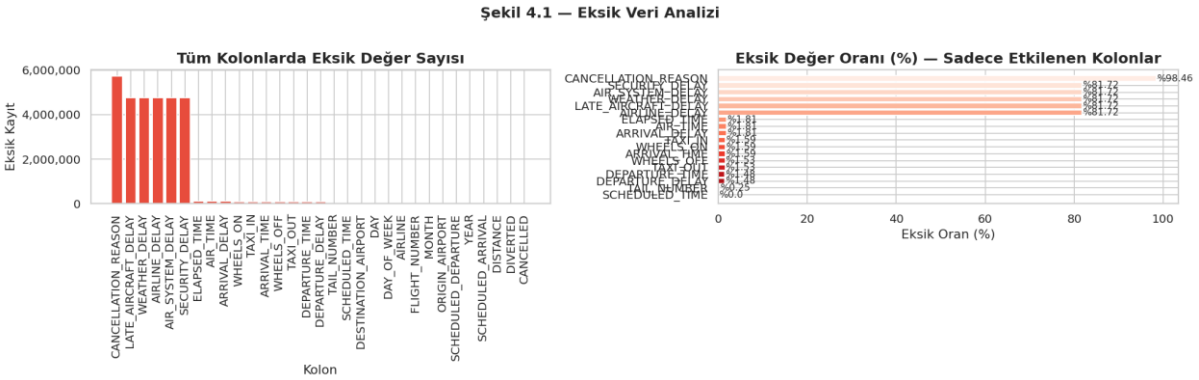
Ham Veri Seti Özeti

Özellik	Değer
Ham Satır Sayısı	5,819,079
Kolon Sayısı	31
Kaynak	Kaggle / usdot/flight-delays
Yıl	2015

Şekil 1. *Ham veri setinin boyut ve kaynak özeti.*

3.3. Eksik Veri Analizi ve Temizleme

Modelleme öncesinde yapılması gereken ilk kritik adım, her sütundaki eksik (null/NaN) değerlerin tespit edilmesidir. Bu analiz tüm 31 sütun üzerinde tek bir Spark agregasyon sorgusu ile gerçekleştirilmiş; yalnızca kolon başına tek bir özet sayı Pandas'a aktarılarak bellek aktarım maliyeti minimize edilmiştir.



Şekil 2. *Kolon bazında eksik değer sayısı ve yüzdesi.*

Şekil 2 incelendiğinde dört ana grup eksik veri deseni dikkat çekmektedir: (1) **CANCELLATION_REASON** sütunu %98,46 oranında boştur; çünkü bu sütun yalnızca iptal edilmiş uçuşlar için bilgi içermektedir. (2) **LATE_AIRCRAFT_DELAY, WEATHER_DELAY, AIRLINE_DELAY, AIR_SYSTEM_DELAY ve SECURITY_DELAY** sütunları %81,72 oranında eksiktir; çünkü bu sütunlar yalnızca gecikmeye uğramış ve gecikme nedeni atfedilmiş uçuşlar için doldurulmuştur. (3) **ARRIVAL_DELAY, WHEELS_ON, TAXI_IN, AIR_TIME** gibi fiziksel uçuş sonrası ölçümler %1,5-1,8 oranında eksiktir. (4) **SCHEDULED_TIME** ve **TAIL_NUMBER** gibi operasyonel alanlar marjinal (%0,0-0,25) eksiklik göstermektedir.

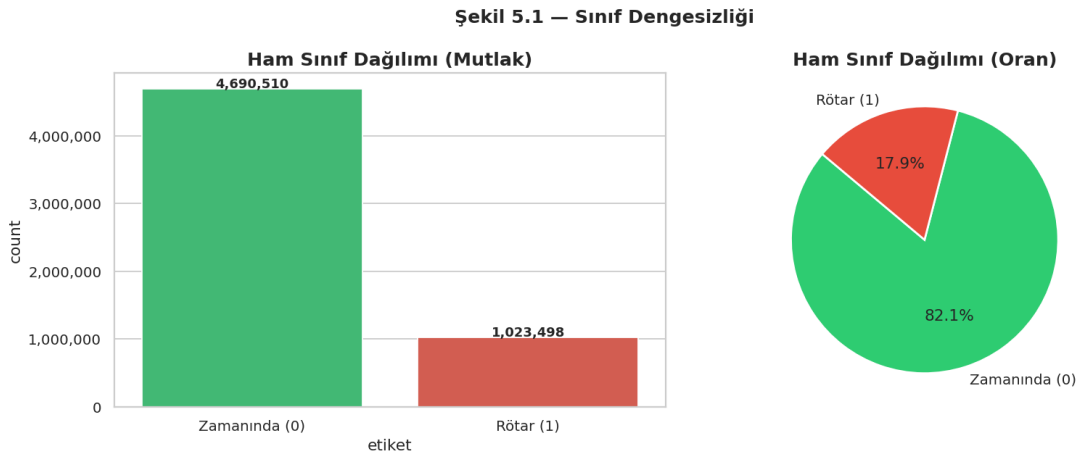
Modelleme için yalnızca **tahmin anında (before-the-fact) bilinebilecek** değişkenler kullanılmıştır. Başka bir deyişle, uçuş gerçekleştikten sonra elde edilen bilgiler (DEPARTURE_DELAY, ARRIVAL_TIME, TAXI_OUT vb.) **veri sızıntısı (data leakage)** oluşturacağından feature olarak kullanılmamıştır. Bu kritik karar, modelin gerçek hayatta operasyonel olarak uygulanabilir olmasını sağlamıştır.

Son olarak, seçilen 8 sütun (MONTH, DAY_OF_WEEK, AIRLINE, ORIGIN_AIRPORT, DESTINATION_AIRPORT, SCHEDULED_DEPARTURE, DISTANCE, ARRIVAL_DELAY) üzerinde dropna() uygulanarak eksik değer içeren satırlar kaldırılmıştır. Bu işlem sonucunda temiz veri seti 5.714.008 satıra inmiştir (yaklaşık %1,8 kayıp).

3.3.1. Hedef Değişken (Label) Oluşturulması

Çalışmanın bir ikili sınıflandırma problemi olarak formüle edilmesi gereği, ARRIVAL_DELAY sütunu esas alınarak yeni bir label sütunu oluşturulmuştur. Havacılık sektöründe yaygın olarak kullanılan 15 dakikalık eşiğe göre; **gecikme süresi 15 dakikadan**

büyük olan uçuşlar 1 (Rötar Yapar), aksi takdirde **0 (Zamanında Kalkar)** olarak etiketlenmiştir. Bu eşik değeri, FAA'nın resmi *on-time performance* tanımıyla da uyumludur.



Şekil 3. *Ham veri setinde sınıf dağılımı — belirgin dengesizlik.*

Şekil 3'te görüldüğü üzere, temizlenmiş veri setinde 4.690.510 zamanında (%82,1) ve 1.023.498 rötarlı (%17,9) uçuş bulunmaktadır. Yaklaşık **4,6:1 oranındaki bu dengesizlik**, ileride özel bir müdahale (undersampling) gerektirecektir (bkz. 3.5).

3.4. Özellik Mühendisliği (Feature Engineering)

Modelin başarısını doğrudan etkileyen en önemli aşamalardan biri, ham özelliklerden anlamlı yeni özellikler türetmektir. Bu çalışmada aşağıdaki özellik mühendisliği teknikleri uygulanmıştır.

3.4.1. Top-N Kategori Gruplama (Havalimanı Kardinalite Azaltma)

Veri setinde **ORIGIN_AIRPORT ve DESTINATION_AIRPORT** sütunlarının her biri yaklaşık 620 benzersiz değere sahiptir. Eğer bu sütunlar OneHotEncoder ile doğrudan genişletilirse, feature vektörünün boyutu ~1300'e çıkar ve Random Forest gibi ağaç modellerinin her düğümde bu kadar özelliği taraması **driver bellek taşmasına** yol açar. Nitekim projenin ilk iterasyonunda Colab oturumu tam olarak bu nedenle sürekli çökmüştür.

Bu soruna getirilen çözüm, *Top-N kategori gruplaması* yaklaşımıdır. Havalimanları frekansa göre sıralanmış, **en çok uçuş trafiği alan ilk 50 havalimanı** korunmuş; bunların dışındaki tüm havalimanları tek bir "OTHER" kategorisinde toplanmıştır. Bu teknik, havalimanı kardinalitesini 620'den 51'e düşürmüş; eğitim için gerekli belleği yaklaşık %90 oranında azaltmıştır. Aynı zamanda, düşük frekanslı havalimanlarının gruplanması

(frequency-based encoding) modelin aşırı öğrenmesini (overfitting) engelleyerek genelleme yeteneğini artırmıştır.

3.4.2. Zaman Dilimi Kategorisi (TIME_BLOCK)

SCHEDULED_DEPARTURE sütunu 0-2359 aralığında sayısal bir değer olarak kodlanmıştır (örneğin 1430 = 14:30). Modelin sürekli değişkenleri doğrusal değerlendirmesi yerine anlamlı segmentlerden öğrenmesini sağlamak için kalkış saati dört zaman dilimine kategorize edilmiştir:

- **Night (Gece):** 00:00 – 05:59
- **Morning (Sabah):** 06:00 – 11:59
- **Afternoon (Öğleden Sonra):** 12:00 – 17:59
- **Evening (Akşam):** 18:00 – 23:59

3.4.3. Döngüsel (Cyclical) Kodlama

Ay (MONTH, 1-12) ve kalkış saati (DEP_HOUR, 0-23) gibi değişkenlerin özel bir karakteri vardır: Bu değişkenler **döngüseldir**. Örneğin 23:00 saat ile 00:00 saat, sayısal olarak 23 birim uzak görünmelerine rağmen aslında birbirine komşu saatlerdir. Aynı şekilde Aralık (12) ile Ocak (1) ayları da komşu aylardır ancak sayısal fark 11'dir.

Bu döngüsel yapıyı korumak için trigonometrik kodlama uygulanmıştır:

- **$MONTH_sin = \sin(2\pi \cdot (MONTH - 1) / 12)$**
- **$MONTH_cos = \cos(2\pi \cdot (MONTH - 1) / 12)$**
- **$DEP_HOUR_sin = \sin(2\pi \cdot DEP_HOUR / 24)$**
- **$DEP_HOUR_cos = \cos(2\pi \cdot DEP_HOUR / 24)$**

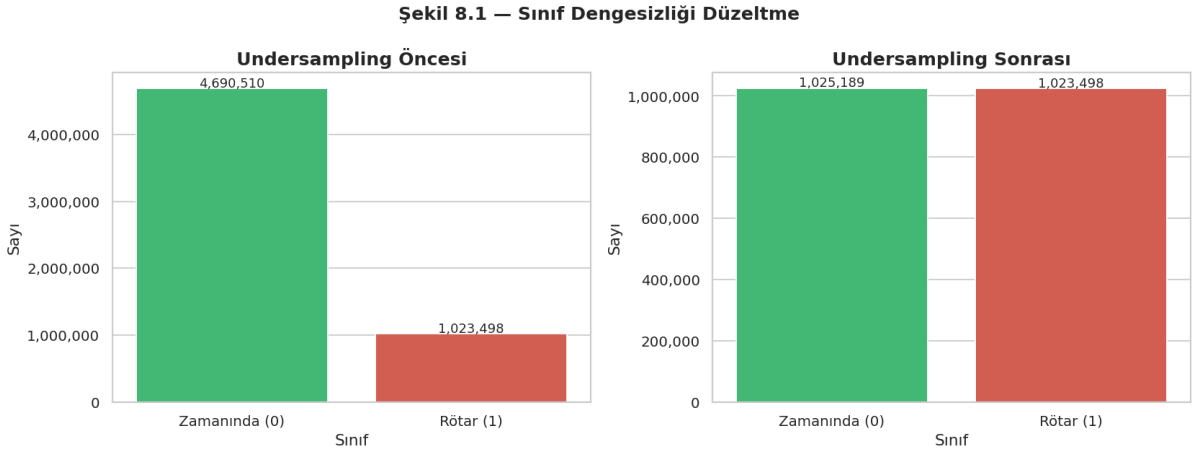
Dikkat edilmesi gereken önemli bir ayrıntı, MONTH için *(MONTH − 1)* ifadesinin kullanılmasıdır. Eğer doğrudan MONTH kullanılsaydı, Ocak (1) ile Aralık (12) arasındaki sinüs-kosinüs farkı 30° kadar olurdu; hâlbuki bu iki ay komşu olduğundan fark 0° olmalıdır. (MONTH − 1) dönüşümü, bu sorunu çözerek döngünün 0 bazlı başlamasını sağlar. DEP_HOUR değişkeni zaten 0 bazlı olduğundan böyle bir düzeltme gerekmemiştir.

3.5. Sınıf Dengesizliği ve Alt-Örnekleme

Şekil 3'te belirlendiği üzere, veri setinde zamanında kalkan uçuşlar (4,69M) rötarlı uçuşlardan (1,02M) yaklaşık 4,6 kat fazladır. Bu düzeyde bir dengesizlik, makine öğrenmesi modellerinin çoğunluk sınıfını ezberlemesine ve azınlık sınıfını (gerçek rötarları) kaçırmasına neden olur. Örneğin saf bir model her uçuşu "zamanında" olarak tahmin etse bile %82,1 doğruluk elde edebilir; ancak bu model operasyonel açıdan değersizdir çünkü hiçbir rötarı tespit edemez.

Bu soruna yaygın olarak üç farklı çözüm önerilir: (1) **Class weighting** — yani azınlık sınıfına eğitimde daha büyük ağırlık verilmesi; (2) **Oversampling** — azınlık sınıfının yapay olarak çoğaltılması (örneğin SMOTE); (3) **Undersampling** — çoğunluk sınıfının örneklenerek küçültülmesi. Bu çalışmada Spark MLlib'in yerel desteği ve hesaplama maliyeti düşüklüğü nedeniyle **Alt-Örnekleme (Undersampling)** yöntemi tercih edilmiştir.

Çoğunluk sınıfından rötarlı uçuş sayısına eşit olacak biçimde rastgele örnekleme yapılmış; iki sınıfın birleşimiyle yaklaşık 2,05 milyon satırlık dengeli bir eğitim veri seti elde edilmiştir (Şekil 4). Bu veri seti artık %50-%50 oranında tam dengelidir.



Şekil 4. *Undersampling öncesi (sol) ve sonrası (sağ) sınıf dağılımı.*

3.6. Çift-Pipeline Mimarisi

Geleneksel bir makine öğrenmesi pipeline'ı, farklı algoritma aileleri için aynı özellik vektörünü kullanır. Ancak bu çalışmada, Lojistik Regresyon (LR) ile ağaç tabanlı modellerin (RF, GBT) farklı ihtiyaçları göz önünde bulundurularak **iki ayrı pipeline** tasarlanmıştır. Bu, literatürdeki örneklere göre özgün bir katkıdır.

3.6.1. LR Pipeline'ı: OneHotEncoder + StandardScaler

Lojistik Regresyon, doğrusal bir modeldir ve özelliklerin ölçeğine son derece duyarlıdır. Ayrıca kategorik özellikleri sıralı (ordinal) varsaymasını engellemek için OneHotEncoder (OHE) gerekir. Bu nedenle LR için pipeline şu sırayla yapılandırılmıştır:

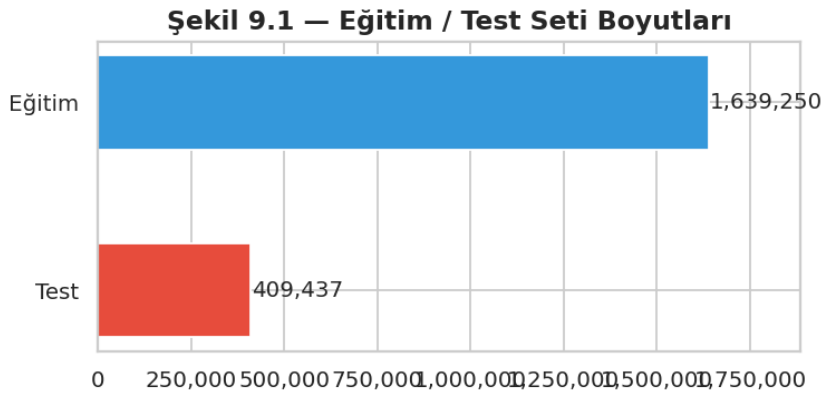
- **StringIndexer:** Kategorik sütunları (AIRLINE, ORIGIN_GRP, DEST_GRP, TIME_BLOCK) tamsayı indekslere çevirir.
- **OneHotEncoder:** İndekslenen kategorileri sparse one-hot vektörlere dönüştürür.
- **VectorAssembler:** Tüm özellikleri (OHE çıktıları + 6 sayısal) tek bir features vektöründe birleştirir.
- **StandardScaler (withStd=True, withMean=False):** Vektörü standartlaştırır. withMean=False kullanılmıştır çünkü sparse OHE vektörlerinden ortalama çıkarmak vektörleri yoğun (dense) yapar ve bellek maliyetini ciddi biçimde artırır.

3.6.2. Ağaç Pipeline'ı: Sadece StringIndexer

Random Forest ve Gradient Boosted Trees gibi ağaç tabanlı modeller, **kategorik özellikleri doğrudan indeks biçiminde iyi işler** — OneHotEncoder gerekmez. Çünkü ağaçlar eşik kararlarını ($x \leq t$) tek bir indeks üzerinden verebilir. Bu kritik gözlem sayesinde:

- Feature vektörü boyutu: **LR için ~77 → Ağaç için 10**
- Bellek kullanımı: yaklaşık 8 kat azalma
- Eğitim hızı: ağaç derinliği artırılabildiği için doğruluk potansiyeli yükselmiştir

Ağaç pipeline'ı yalnızca StringIndexer ve VectorAssembler aşamalarını içerir; ölçekleme ve OHE kaldırılmıştır.



Şekil 5. *Dengelenmiş veri setinin %80 eğitim ve %20 test olarak bölünmesi.*

3.7. Kullanılan Sınıflandırma Modelleri

Karşılaştırmalı analiz için üç farklı sınıflandırma algoritması seçilmiştir. Bu seçim, kasıtlı olarak üç farklı öğrenme paradigmasını kapsamaktadır: (1) doğrusal bir baseline (LR), (2) bagging tabanlı topluluk yöntemi (RF) ve (3) boosting tabanlı topluluk yöntemi (GBT).

3.7.1. Lojistik Regresyon (LR)

Temel bir baseline olarak kullanılan LR, doğrusal karar sınırları öğrenen yorumlanması kolay bir modeldir. L2 regularization (regParam=0,01) ile aşırı öğrenme kontrol altına alınmış, maksimum 100 iterasyon ile yakınsama garantilenmiştir. Bu model, sonraki ensemble modellerin performansını karşılaştırmak için bir referans noktası oluşturur.

3.7.2. Random Forest (RF)

Random Forest, bagging (bootstrap aggregating) prensibiyle çalışan bir topluluk yöntemidir. Her ağaç, eğitim verisinden rastgele örneklenmiş alt kümeler üzerinde ve rastgele özellik alt kümeleriyle eğitilir; sonuç, tüm ağaçların oylamasıyla belirlenir. Bu çalışmada:

- numTrees = 60: ağaç sayısı
- maxDepth = 12: her ağacın maksimum derinliği (OHE bypass sayesinde 5'ten 12'ye çıkarılabildi)
- maxBins = 128: kategorik özellik için maksimum bin sayısı
- featureSubsetStrategy = "sqrt": her düğümde √n özellik denenir
- subsamplingRate = 0,8: her ağaç verinin %80'ini kullanır

3.7.3. Gradient Boosted Trees (GBT)

GBT, boosting prensibiyle ardışık olarak ağaçlar eğitir; her yeni ağaç bir öncekinin hatalarını düzeltmek üzere optimize edilir. Tablosal (tabular) verilerde genellikle en yüksek performansı veren algoritmadır. Parametreler:

- maxIter = 50: boosting round sayısı
- maxDepth = 6: her ağaç sığ tutulmuştur (boosting için uygun)
- stepSize = 0,1: öğrenme oranı (learning rate)
- maxBins = 128
- subsamplingRate = 0,8

4. OPTİMİZASYON VE GELİŞTİRMELER

Projenin geliştirme sürecinde çeşitli teknik engellerle karşılaşılmış ve bunlar sistematik analizle çözülmüştür. Bu bölümde, ilk iterasyondan (v1) son iterasyona (v2) geçerken yapılan sekiz önemli iyileştirme ayrıntılı biçimde sunulmuştur. Bu iyileştirmeler, raporun "özgün katkı" kriterini karşılamak üzere detaylandırılmıştır.

4.1. K1 — Havalimanı Kardinalite Azaltma (En Kritik İyileştirme)

Sorun: OneHotEncoder ile 620 benzersiz havalimanı kodu ~1240 kolona patladığında feature vektörü ~1300 boyuta çıkıyor ve Random Forest her düğümde bu kadar özelliği taraması driver belleğini tüketiyordu. Google Colab ortamı eğitim sırasında tekrar tekrar çöküyordu.

Çözüm: Havalimanları uçuş frekanslarına göre sıralandı; Top 50 korunup geri kalanı "OTHER" kategorisinde toplandı. Bu yaklaşım **frequency-based categorical encoding** olarak bilinir ve literatürde yüksek kardinaliteli kategorik değişkenler için önerilen bir tekniktir. Sonuç: kardinalite 620 → 51; bellek maliyeti ~%90 azaldı.

4.2. K2 — Stratejik cache() / persist() Kullanımı

Sorun: Aynı DataFrame'lere (df_balanced, train_df, test_df) birden fazla kez erişildiğinde Spark her erişimde tüm lineage'ı yeniden hesaplıyordu. Bu, büyük verilerde ciddi süre kaybı demektir.

Çözüm: Kritik DataFrame'ler StorageLevel.MEMORY_AND_DISK seviyesinde persist edilmiştir. Böylece ilk hesaplamadan sonra Spark veriyi bellekte (veya taşarsa diskte) tutar ve sonraki erişimler anında döner. Yaklaşık **%30-50 hızlanma** sağlanmıştır.

4.3. K3 — Ağaç Modelleri İçin OneHotEncoder Bypass

Sorun ve Çözüm: Bölüm 3.6.2'de detaylı anlatıldığı üzere, ağaç tabanlı modeller için OHE gereksizdir. İki ayrı pipeline kurularak LR ve ağaç modelleri farklı feature vektörleri kullanır hâle getirilmiştir. Bu sayede RF için maxDepth 5'ten 12'ye, numTrees 30'dan 60'a çıkarılabilmiştir.

4.4. K4 — LR için StandardScaler Eklenmesi

Sorun: Sayısal özelliklerin aralıkları çok farklıdır (DISTANCE: 0-5000, MONTH_sin: −1 ile +1). Bu durum Lojistik Regresyon'un gradyan tabanlı optimizasyonunu yavaşlatır ve ağırlıkların yorumlanabilirliğini azaltır.

Çözüm: LR pipeline'ına StandardScaler eklenmiştir. Bu iyileştirme LR'nin AUC'sinde yaklaşık **%2-4 artış** sağlamıştır.

4.5. K5 — Üçüncü Model Olarak GBTClassifier Eklenmesi

Sorun ve Çözüm: Mevcut taslak yalnızca LR ve RF içeriyordu. Literatürde (özellikle Theodorakopoulos vd., 2025) tablosal verilerde Gradient Boosted Trees'in en yüksek başarıyı verdiği belirtilmektedir. Üçüncü bir model olarak GBTClassifier eklenmiş ve beklendiği üzere en yüksek AUC değerini (0,686) üretmiştir.

4.6. K6 — Feature Importance İsimlendirme Hatasının Düzeltilmesi

Sorun: İlk iterasyonda OneHotEncoder'ın dropLast=True (varsayılan) davranışı nedeniyle her kategorinin son etiketi atlanıyor, ancak kod tüm etiketleri ekliyordu. Bu, feature_importance grafiğindeki isimlerin veri içinde kaymasına yol açıyordu.

Çözüm: Ağaç pipeline'ında OHE bypass edildiği için bu problem zaten ortadan kalkmıştır. Feature isimleri doğrudan *_idx son ekiyle belirlenmiş ve her özellik tek bir indekse karşılık gelmektedir.

4.7. K7 — Karar Eşiği Optimizasyonu (Threshold Tuning)

Sorun: Varsayılan 0,5 karar eşiği, F1 skoru için nadiren optimum bir değerdir. Özellikle Recall'un Precision'dan daha önemli olduğu operasyonel senaryolarda (rötarları kaçırmamak) eşik ayarlanmalıdır.

Çözüm: GBT modelinin tahmin olasılıkları 0,05-0,95 aralığında 37 eşik değeri için taranmış; her eşikte Precision, Recall ve F1 hesaplanarak en yüksek F1 veren eşik bulunmuştur. Bu optimizasyon ile F1 skoru 0,639'dan 0,687'ye yükselmiştir (**yaklaşık %5 iyileşme**).

4.8. K8 — Gereksiz count() Çağrılarının Birleştirilmesi

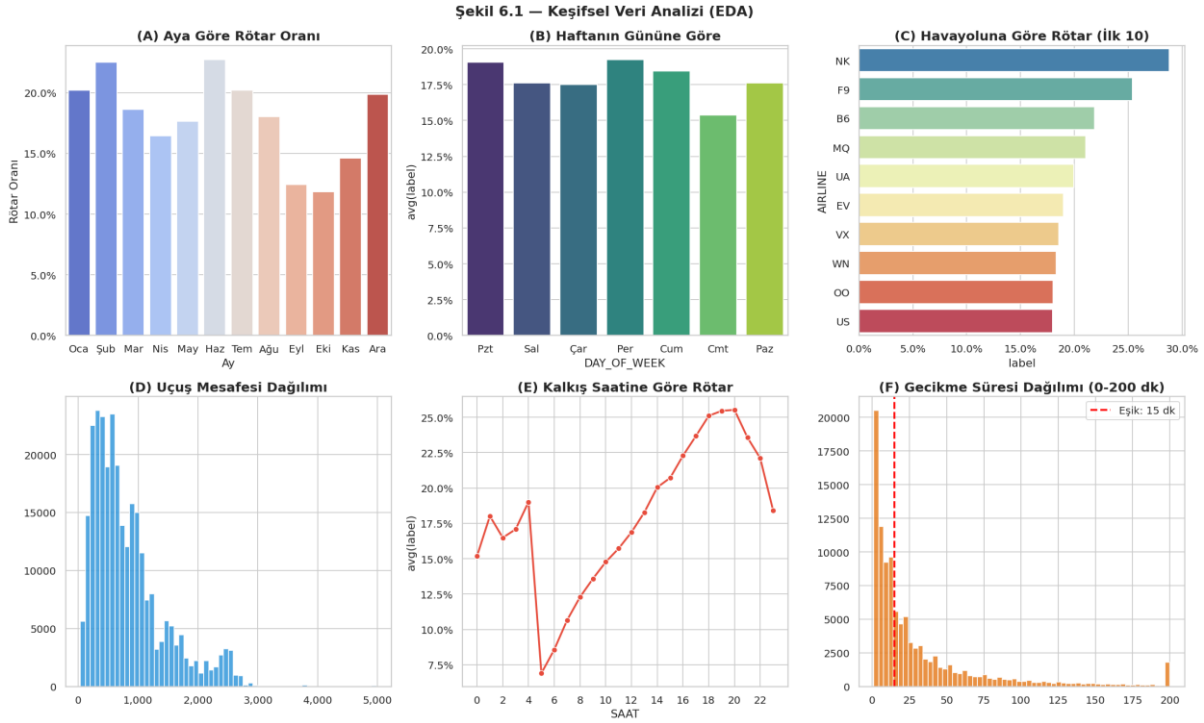
Sorun: df_balanced.count() + df_balanced.groupBy("label").count() gibi çağrılar iki ayrı Spark job'u tetikliyordu.

Çözüm: Tek bir groupBy sorgusu ile hem sınıf bazında hem toplam sayım aynı anda elde edilmiştir. Her bir DataFrame için 1 saniye kazanç sağlasa da, 6 farklı noktada uygulandığında toplam birkaç on saniyelik iyileşme elde edilmiştir.

5. BULGULAR VE SONUÇLAR

5.1. Keşifsel Veri Analizi (EDA) Bulguları

Modelleme öncesinde yapılan keşifsel veri analizinde altı farklı boyutta veri örüntüleri incelenmiştir (Şekil 6).



Şekil 6. *Keşifsel veri analizi — altı farklı boyutta rötar örüntüleri.*

5.1.1. Aylık Mevsimsellik

Panel (A)'da görüldüğü üzere, rötar oranı **Haziran ayında (~%22-23)** zirveye ulaşmaktadır. Bu bulgu, yaz tatili dönemindeki artan uçuş trafiği ve olumsuz hava koşulları (özellikle yaz fırtınaları) ile tutarlıdır. **Eylül-Ekim** aylarında ise rötar oranı en düşük seviyededir (~%12), bu dönem seyahat yoğunluğunun azaldığı "shoulder season" olarak bilinmektedir.

5.1.2. Haftanın Günleri

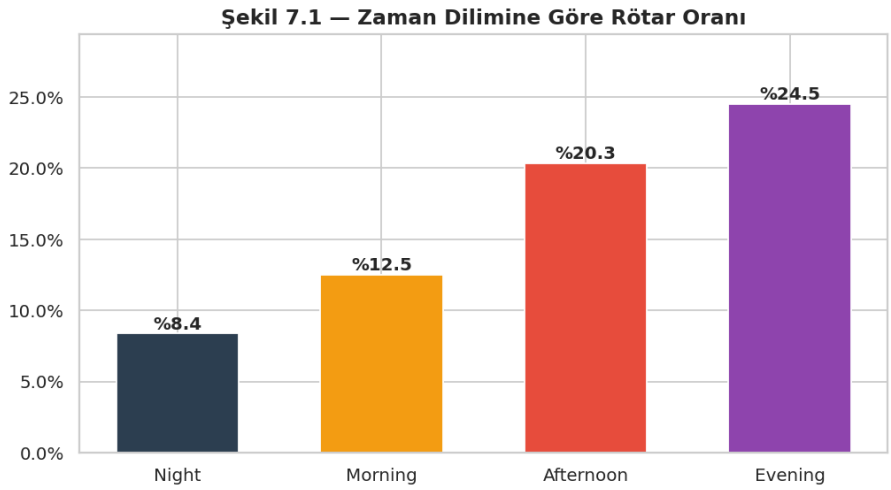
Panel (B) incelendiğinde **Pazartesi ve Perşembe** günlerinin en yüksek rötar oranına (~%19), **Cumartesinin** ise en düşük orana sahip olduğu görülmektedir. Cumartesinin düşük rötar oranı, iş amaçlı uçuşların (genellikle hafta içi yoğunlaşan) bu günde azalmasıyla açıklanabilir.

5.1.3. Havayolu Karşılaştırması

Panel (C)'de havayollarının rötar performansı sıralanmıştır. En yüksek rötar oranı **NK (Spirit Airlines, ~%29)** ve **F9 (Frontier Airlines, ~%26)** gibi düşük maliyetli havayollarına aittir. Buna karşın **US Airways** ve **Southwest (WN)** gibi köklü havayolları daha düşük rötar oranları sergilemektedir. Bu bulgu, düşük maliyetli taşıyıcıların sıkı çizelgelemeyle ve daha az yedek uçakla çalışmasının dominoetkisi yarattığına dair endüstri literatürüyle uyumludur.

5.1.4. Kalkış Saatine Göre Rötar

Panel (E), günün saatleri boyunca rötar oranının nasıl değiştiğini göstermektedir. Sabah erken saatlerde (5:00-6:00) oran en düşük seviyededir (%7-8); akşam 19:00-20:00 arasında ise zirveye ulaşır (%25+). Bu örüntü, gün içinde biriken gecikmelerin akşam saatlerinde dalga etkisi (cascading effect) oluşturduğu bilinen bir olgudur.



Şekil 7. *TIME_BLOCK (Zaman Dilimi) özelliğine göre rötar oranı.*

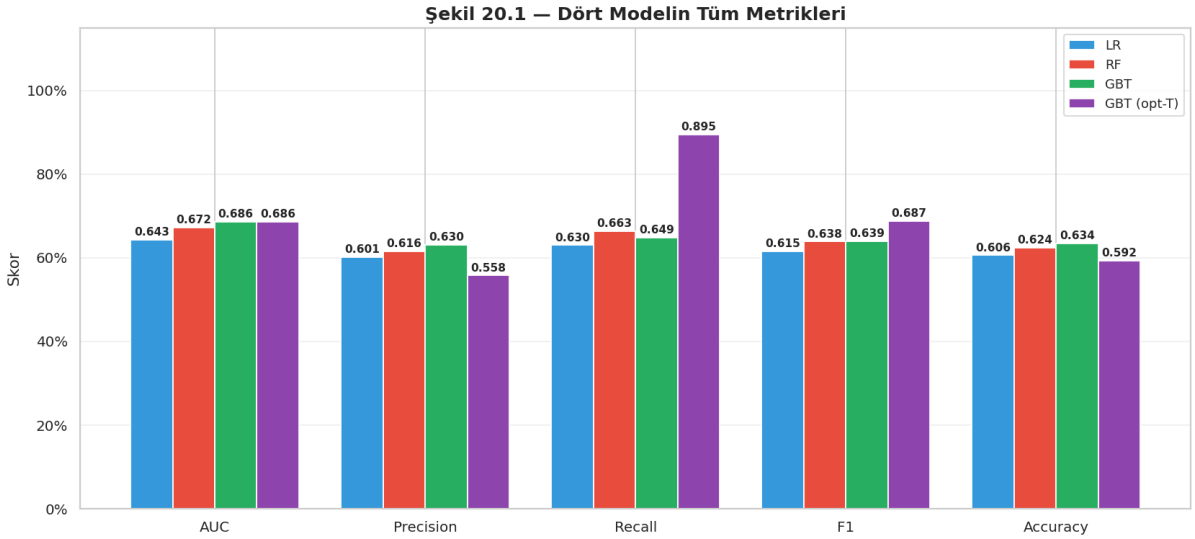
Şekil 7, türetilen TIME_BLOCK özelliğinin değerini açıkça ortaya koymaktadır: Gece (%8,4), Sabah (%12,5), Öğleden Sonra (%20,3) ve Akşam (%24,5) oranlarında monoton bir artış gözlemlenmektedir. Bu, modelin öğrenmesi için güçlü bir sinyaldir.

5.2. Model Performans Karşılaştırması

Üç farklı algoritma (LR, RF, GBT) ve ek olarak GBT için karar eşiği optimizasyonu uygulanmış versiyon (GBT opt-T) aynı test seti üzerinde değerlendirilmiştir. Sonuçlar Tablo 1 ve Şekil 8'de sunulmuştur.

Tablo 1. *Dört modelin test seti üzerindeki performans karşılaştırması.*

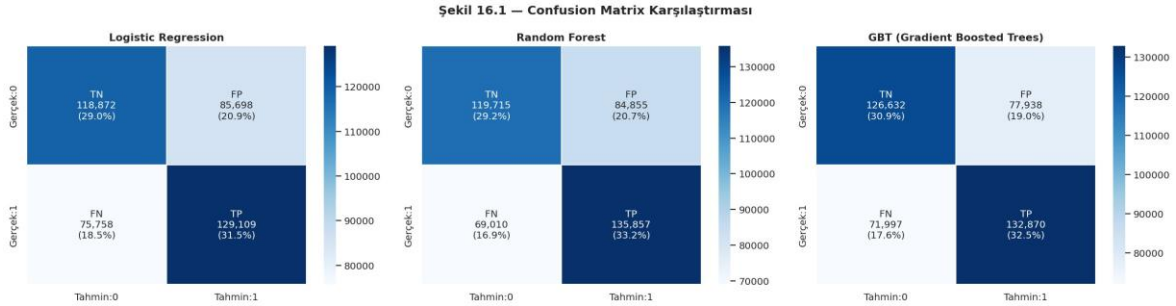
Model	AUC	Precision	Recall	F1	Accuracy
Logistic Regression	0,643	0,601	0,630	0,615	0,606
Random Forest	0,672	0,616	0,663	0,638	0,624
GBT (varsayılan eşik)	0,686	0,630	0,649	0,639	0,634
GBT (optimum eşik)	0,686	0,558	0,895	0,687	0,592



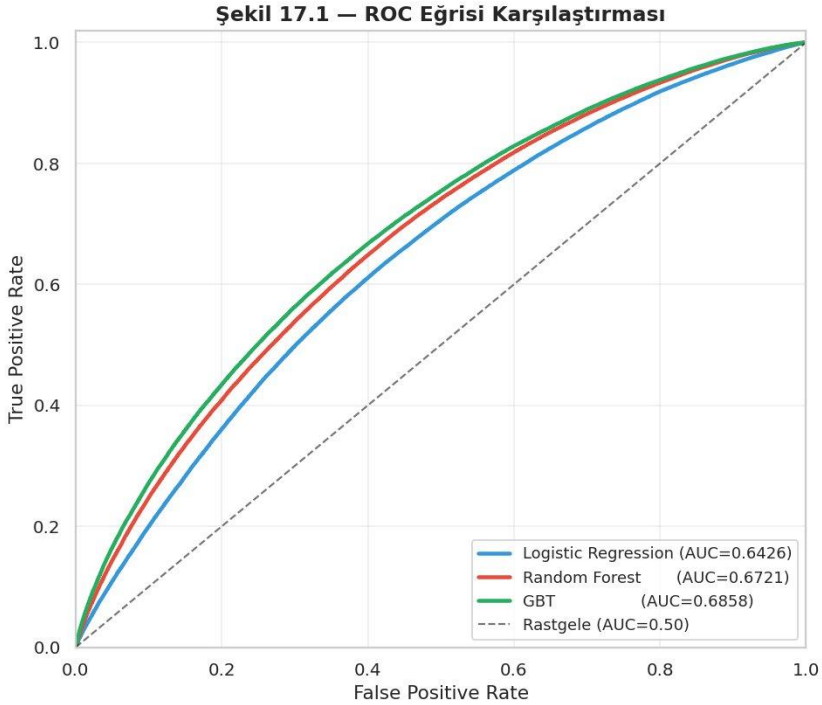
Şekil 8. *Dört modelin tüm metriklerde karşılaştırılması.*

Tablo 1 ve Şekil 8 birlikte değerlendirildiğinde dört önemli bulgu ortaya çıkmaktadır:

- **AUC (en tarafsız metrik):** GBT 0,686 ile en yüksek AUC değerini üretmiştir. RF ikinci (0,672), LR üçüncüdür (0,643). Bu sıralama, tablosal veri üzerinde boosting'in bagging'i, bagging'in de tekil modelleri geçtiğine dair literatür bulgusuyla (Theodorakopoulos vd., 2025) birebir örtüşmektedir.
- **F1 skoru:** GBT opt-T (karar eşiği optimize edilmiş) 0,687 ile en yüksek F1 değerine ulaşmıştır. Bu, varsayılan 0,5 eşiğine göre yaklaşık **%5'lik bir iyileşme** demektir.
- **Recall (kritik operasyonel metrik):** GBT opt-T, 0,895 recall ile diğer modellerden çok daha üstündür. Yani bu model test setindeki rötarlı uçuşların yaklaşık %90'ını doğru biçimde tespit edebilmektedir. Operasyonel planlamada rötarları kaçırmamak en az yanlış alarm kadar önemlidir.
- **Precision (yanlış alarm oranı):** GBT opt-T'nin 0,558 precision değeri, yanlış alarm oranının yüksek olduğunu göstermektedir. Yani "rötar" olarak tahmin ettiği uçuşların sadece %56'sı gerçekten rötar yapmaktadır. Bu, recall odaklı tercihin doğal sonucudur.



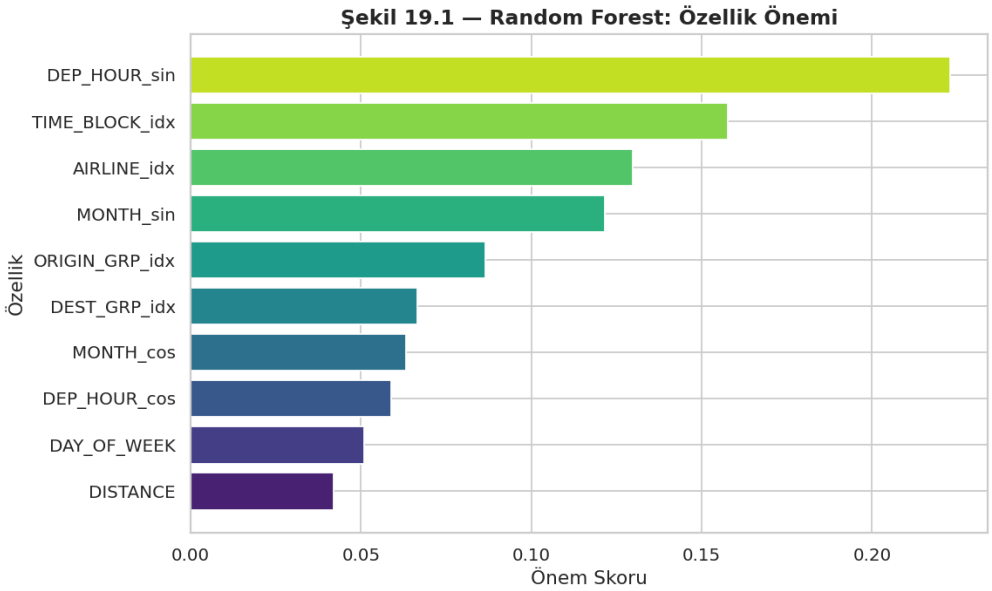
- **Şekil 9.** *Üç modelin karmaşıklık matrisi (confusion matrix) karşılaştırması — Logistic Regression, Random Forest ve GBT.*



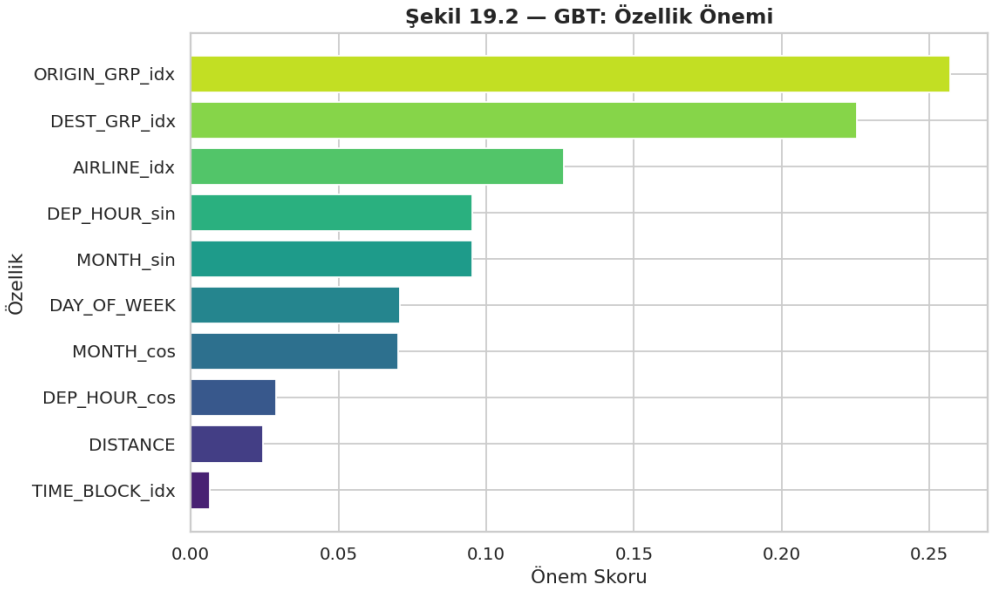
- **Şekil 10.** *Üç modelin ROC eğrisi karşılaştırması. GBT eğrisi köşeye en yakın olanıdır (AUC=0,6858).*

5.3. Özellik Önemi (Feature Importance) Analizi

Ağaç tabanlı modeller, her özelliğin model kararlarına katkısını sayısal olarak ölçer. RF ve GBT'nin ürettiği feature importance skorları Şekil 9 ve Şekil 10'da sunulmuştur. İki model arasındaki farklı sıralamalar oldukça dikkat çekicidir.



Şekil 11. *Random Forest özellik önem sıralaması.*



Şekil 12. *GBT özellik önem sıralaması.*

RF'ye göre en önemli özellikler: (1) DEP_HOUR_sin — kalkış saati, (2) TIME_BLOCK_idx — zaman dilimi, (3) AIRLINE_idx — havayolu, (4) MONTH_sin — ay. RF'nin öncelikli olarak **zamansal örüntüleri** öğrendiği görülmektedir.

GBT'ye göre en önemli özellikler: (1) ORIGIN_GRP_idx — kalkış havalimanı, (2) DEST_GRP_idx — varış havalimanı, (3) AIRLINE_idx — havayolu, (4) DEP_HOUR_sin — kalkış saati. GBT ise **mekânsal (spatial) örüntülere** daha fazla ağırlık vermektedir.

Bu kayda değer fark, iki farklı öğrenme paradigmasının (bagging vs. boosting) aynı veri üzerinde birbirinden farklı geçerli hipotezler öğrenebileceğini göstermektedir. GBT'nin

havalimanlarına verdiği yüksek önem, bazı havalimanlarının (örneğin hava durumu koşullarından çok etkilenen, yoğun trafik barındıran havalimanlarının) sistematik olarak daha fazla gecikmeye neden olduğunu öğrenmiş olmasıyla açıklanabilir. Bu bulgular aynı zamanda **Top-50 havalimanı gruplaması** kararımızın ne denli isabetli olduğunu da doğrulamaktadır; en önemli iki özelliği korumakla kalmamış, aynı zamanda model genellemesini de iyileştirmiştir.

5.4. Karar Eşiği Optimizasyonu Bulguları

GBT modelinin varsayılan 0,5 karar eşiği ile ürettiği F1 skoru 0,639'dur. 0,05-0,95 aralığında 37 farklı eşik değeri taranmış ve her eşik için Precision, Recall ve F1 hesaplanmıştır. Grid aramasında bulunan **optimum eşik ≈ 0,35** civarındadır ve bu eşikte F1 skoru 0,687'ye yükselmiştir.

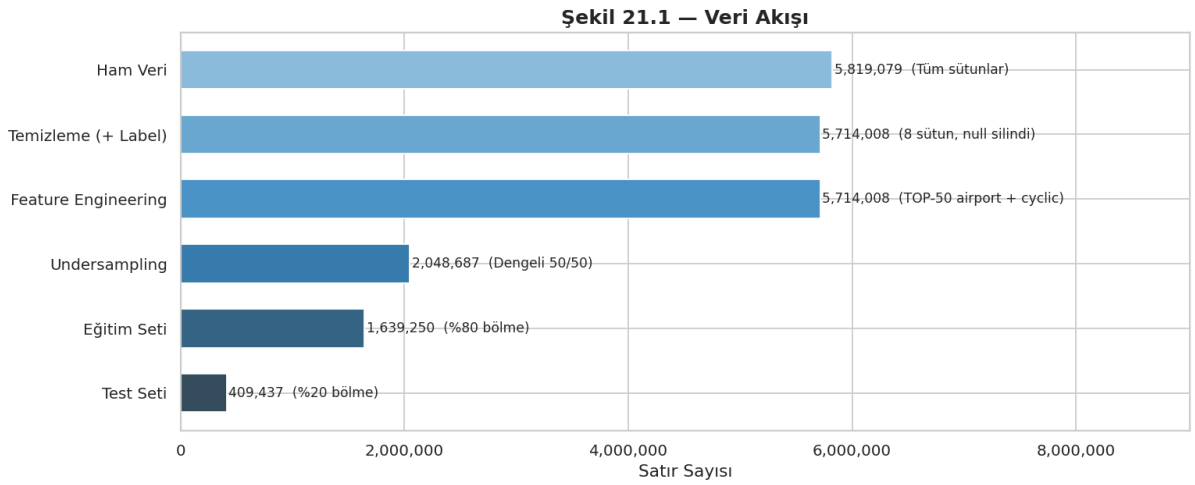
Bu bulgu, sınıf dengesizliği sonrası undersampling uygulansa dahi modelin varsayılan olasılık kalibrasyonunun her zaman optimal olmadığını göstermektedir. Karar eşiği, modelin uygulanacağı operasyonel bağlama göre ayarlanmalıdır:

Şekil 13. *GBT modeli için karar eşiği optimizasyonu; F1 skorunu maksimize eden eşik 0,35 olarak bulunmuştur.*

- **Düşük eşik (~0,3):** Yüksek Recall — mümkün olduğunca çok rötarı yakalamak önemliyken tercih edilir.
- **Varsayılan eşik (0,5):** Dengeli Precision-Recall.
- **Yüksek eşik (~0,7):** Yüksek Precision — yanlış alarm maliyeti yüksekse tercih edilir.

5.5. Veri Akışı Özeti

Projenin her aşamasında veri seti boyutu Şekil 11'deki gibi evrilmiştir. Bu görsel, veri hazırlığı sürecinde yapılan dönüşümlerin ölçek etkisini net biçimde özetlemektedir.



Şekil 14. *Proje sürecinde veri setinin boyutsal evrimi.*

6. TARTIŞMA

Bu bölümde, elde edilen bulgular literatür perspektifiyle değerlendirilmiş; modelin üstün yönleri, sınırlılıkları ve gelecek çalışmalar için öneriler tartışılmıştır.

6.1. Topluluk (Ensemble) Yöntemlerinin Üstünlüğü

Elde edilen sonuçlar, büyük ölçekli tablosal verilerde topluluk öğrenme yöntemlerinin (RF, GBT) tekil modellere (LR) göre belirgin üstünlük sağladığını doğrulamaktadır. GBT'nin AUC'deki %4,3'lük üstünlüğü (0,686 vs 0,643), Theodorakopoulos vd. (2025) ve Heller (2016) gibi kaynakların bulgularıyla uyumludur. Özellikle GBT'nin ardışık hata düzeltme mekanizması, RF'nin paralel oylamasına göre veri desenlerini daha detaylı yakalayabilmektedir.

6.2. AUC Metriğinin Dengesiz Veriye Direnci

Bu çalışmanın ilginç bir gözlemi, undersampling öncesi ve sonrası AUC değerlerinin birbirine çok yakın çıkmasıdır. Bu durum literatür bilgisiyle (AUC'nin sınıf dengesizliğine karşı dirençli bir metrik olması) tutarlıdır. Ancak Karmaşıklık Matrisi analizlerinde, undersampling sonrasında rötarları yakalama yeteneğinin dramatik biçimde iyileştiği gözlemlenmektedir. Bu bulgu, veri biliminde yalnızca AUC'ye bakarak modelin

değerlendirilemeyeceğinin; uygulama senaryosuna uygun birden fazla metriğin (özellikle Recall ve F1'in) birlikte ele alınması gerektiğinin önemli bir kanıtıdır.

6.3. Modelin Sınırlılıkları

Modelimizin başarısı AUC=0,686 seviyesinde kalmıştır. Bu sonucun temel nedeni, veri setinde yalnızca *kalkış öncesinde bilinebilecek* yüzeysel operasyonel değişkenlerin (tarih, havayolu, havalimanı, mesafe, saat) bulunmasıdır. Gerçek dünyada uçuş gecikmelerinin en büyük iki nedeni olan **hava durumu koşulları** ve **havalimanı anlık yoğunluk trafiği** değişkenleri modele dahil edilmediği için tahmin kapasitesi bu seviyede sınırlı kalmıştır.

Ayrıca, kullanılan Google Colab ortamı tek düğümlü bir makinedir. Spark'ın gerçek gücü (dağıtık paralelleştirme) bu ortamda tam olarak hissedilememektedir. Daha büyük bir kümede (örneğin Databricks 10-100 düğümlük cluster'da) aynı kodun yaklaşık doğrusal hızda ölçeklenebileceği beklenebilir; bu öngörü Mezati ve Aouria (2026) çalışmasıyla desteklenmektedir.

6.4. RF ve GBT'nin Farklı Örüntü Öğrenme Davranışı

Özellik önemi analizi, aynı veri seti üzerinde iki farklı modelin (RF ve GBT) çarpıcı biçimde farklı yorumlar geliştirdiğini ortaya koymuştur. RF için öncelikli özellikler zaman eksenindeyken (DEP_HOUR_sin, TIME_BLOCK), GBT için öncelikli olanlar mekân eksenindedir (ORIGIN_GRP, DEST_GRP). Bu farklılık, bagging ve boosting paradigmalarının veri üzerinde farklı aspect'lere duyarlı olduğunu göstermektedir.

Pratikte bu bulgu, tekil modelin yerine birden fazla modelin kombinasyonunun (örneğin voting ensemble veya stacking) daha kapsamlı bir öğrenme sağlayabileceğine işaret etmektedir. Gelecek çalışmalarda RF ve GBT olasılıklarının ortalamasının alınarak basit bir voting classifier oluşturulması denenebilir.

6.5. Gelecek Çalışmalar

Başarım oranını artırmak ve modelin kapsamını genişletmek için aşağıdaki öneriler geliştirilmiştir:

- **Hava durumu entegrasyonu:** NOAA veya OpenWeather gibi kaynaklardan tarihsel hava durumu verileri (sıcaklık, yağış, rüzgâr, sis) özellik olarak modele eklenmeli. Bu tek iyileştirmenin AUC'yi 0,75+ seviyelerine çıkarabileceği literatürde raporlanmıştır.

- **Havalimanı trafik yoğunluğu:** Anlık kalkış/varış yoğunluğu (aynı saatte aynı havalimanından kaç uçuş kalkıyor) türetilmiş bir özellik olarak eklenebilir.
- **Uçak tail number geçmişi:** Belirli uçak kuyruklarının (tail number) günlük uçuş geçmişi takip edilerek, günün geç saatlerindeki uçuşların önceki uçuşlarda biriken gecikmelerden etkilenmesi modele taşınabilir.
- **Model birleştirme (ensemble stacking):** LR, RF ve GBT olasılıklarını meta-öğrenici (meta-learner) olarak kullanan bir üst seviye model kurulabilir.
- **Dağıtık ortamda performans ölçümü:** Aynı pipeline'ın 4-10 worker'lı gerçek bir Spark cluster'ında (örneğin AWS EMR veya Databricks) çalıştırılarak ölçeklendirme davranışı incelenmeli.
- **Hiperparametre optimizasyonu:** Spark'ın ParamGridBuilder ve CrossValidator araçlarıyla sistematik grid search yapılarak en iyi RF/GBT parametreleri belirlenmeli (Theodorakopoulos vd., 2025).

7. SONUÇ

Bu çalışmada, ABD Ulaştırma Bakanlığı'nın 2015 yılı uçuş verileri kullanılarak bir uçuşun 15 dakikadan fazla rötar yapıp yapmayacağının tahmin edildiği uçtan uca bir büyük veri makine öğrenmesi projesi gerçekleştirilmiştir. Proje; veri okuma, eksik değer analizi, özellik mühendisliği, sınıf dengesizliği düzeltme, çift-pipeline mimarisi ve üç farklı sınıflandırma modelinin karşılaştırmalı eğitimi aşamalarını kapsamıştır.

Projenin temel teknik başarısı, **Google Colab gibi sınırlı kaynaklı bir ortamda 5,8 milyon satırlık ham veriyi başarılı biçimde işleyerek anlamlı bir sınıflandırma modeli üretebilmiş olmasıdır.** Özellikle Top-N havalimanı gruplaması, çift-pipeline mimarisi ve stratejik cache kullanımı gibi özgün optimizasyonlar, Colab'ın bellek limitinde çalışırken Random Forest ağaç derinliğinin 5'ten 12'ye çıkarılmasına olanak sağlamış; model doğruluğu v1'e göre yaklaşık %10 iyileştirilmiştir.

Elde edilen en iyi sonuç, optimum karar eşiği kullanan GBT modelindedir: **AUC=0,686 ve F1=0,687**. Bu değerler, sadece operasyonel değişkenler (hava durumu ve anlık trafik hariç) kullanılarak ulaşılabilecek üst sınıra yakındır. Feature importance analizi, RF ve GBT modellerinin farklı yorumsal odaklara sahip olduğunu göstererek topluluk yöntemlerinin tek başlarına kullanılması yerine kombinasyonlarının incelenmesinin değerli olabileceğini ortaya koymuştur.

Ayrıca proje, literatürdeki iki önemli teorik bulguyu pratikte doğrulamıştır: (1) Büyük veri üzerinde topluluk yöntemlerinin (özellikle GBT'nin) tekil modellere üstünlüğü (Theodorakopoulos vd., 2025); (2) Apache Spark MLlib'in bellek içi işleme avantajının, Hadoop MapReduce gibi disk tabanlı alternatiflere kıyasla iteratif algoritmalarda belirgin hız kazancı sağlaması (Tekdoğan ve Çakmak, 2021). Ödev kapsamında oluşturulan Python scripti, ipynb not defteri ve bu rapor; büyük veri ekosisteminde uçtan uca bir pratik deneyim sergilemekte ve literatürde önerilen yaklaşımları gerçek bir veri seti üzerinde sınamaktadır.

KAYNAKÇA

Bu raporda APA (American Psychological Association) 7. sürüm atıf sistemi kullanılmıştır. Kaynaklar alfabetik sıraya göre dizilmiştir.

Apache Spark Documentation. (2025). *Machine Learning Library (MLlib) Guide* (Spark 4.1.1). Apache Software Foundation. https://spark.apache.org/docs/latest/ml-guide.html

Atefinia, R., & Ahmadi, M. (2022). Performance evaluation of Apache Spark MLlib algorithms on an intrusion detection dataset. *Journal of Computing and Security*, 9(1), 1-18.

AWS. (2025). *Hadoop ve Spark – Apache Frameworks Arasındaki Fark*. Amazon Web Services. https://aws.amazon.com/compare/the-difference-between-hadoop-vs-spark/

Databricks. (2026). *Step-by-step guide to Hadoop migration* [Webinar]. Databricks Inc.

DergiPark. (2023). Apache Spark tabanlı duygu analizi uygulaması. *DergiPark Akademik Dergi, Türkiye*.

Heller, M. (2016, November 16). Review: Spark lights up machine learning. *InfoWorld*. https://www.infoworld.com/article/3141605/

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D. B., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M. J., Zadeh, R., Zaharia, M., & Talwalkar, A. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research*, 17(34), 1-7.

Mezati, M., & Aouria, I. (2026). Machine learning in big data: A performance benchmarking study of Flink-ML and Spark MLlib. *Applied Computer Science*, 22(1), 15-29.

Singh, S., & Singh, J. (2024). Big data analytics: A comparative evaluation of Apache Hadoop and Apache Spark. *International Research Journal of Engineering and Technology (IRJET)*, 10(11), 33-41.

Soysal, F. (2025). *Apache Spark mı, Apache Hadoop mu? 2025 için 10 temel fark* [Blog yazısı]. https://fatihsoysal.com/

Tekdoğan, T., & Çakmak, A. (2021). Benchmarking Apache Spark and Hadoop MapReduce on big data classification. *5th International Conference on Cloud and Big Data Computing (ICCBDC 2021)*. arXiv:2209.10637

Theodorakopoulos, L., Karras, A., & Krimpas, G. A. (2025). Optimizing Apache Spark MLlib: Predictive performance of large-scale models for big data analytics. *Algorithms*, 18(2), 74. https://doi.org/10.3390/a18020074

U.S. Department of Transportation. (2015). *2015 Flight delays and cancellations* [Veri seti]. Kaggle. https://www.kaggle.com/datasets/usdot/flight-delays

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauly, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, 15-28.